

UW PICO 5.09

File: signal_handler.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 32
#define BUF_SIZE 1024

volatile pid_t foreground_pid = 0;

// Trim leading/trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Custom SIGINT (Ctrl+C) Signal Handler
void sigint_handler(int sig) {
    if (foreground_pid > 0) {
        // Forward SIGINT directly to the running foreground process
        kill(foreground_pid, SIGINT);
        printf("\n[Signal Handler] Forwarded SIGINT to foreground process (PID: %d)\n", foreground_pid);
    } else {
        // Protect shell: redisplay prompt without exiting
        printf("\n[Signal Handler] SIGINT caught. Shell protected. (No foreground job)\n");
        printf("SignalShell > ");
        fflush(stdout);
    }
}

// Register and configure signal handlers using sigaction
void setup_signal_handlers() {
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART; // Automatically restart interrupted system calls like fgets

    if (sigaction(SIGINT, &sa, NULL) < 0) {
        perror("sigaction registration failed");
        exit(EXIT_FAILURE);
    }
}
```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Tex

UW PICO 5.09

File: signal_handler.c

```
void sigint_handler(int sig) {
    if (foreground_pid > 0) {
        // Forward SIGINT directly to the running foreground process
        kill(foreground_pid, SIGINT);
        printf("\n[Signal Handler] Forwarded SIGINT to foreground process (PID: %d)\n", foreground_pid);
    } else {
        // Protect shell: redisplay prompt without exiting
        printf("\n[Signal Handler] SIGINT caught. Shell protected. (No foreground job)\n");
        printf("SignalShell > ");
        fflush(stdout);
    }
}

// Register and configure signal handlers using sigaction
void setup_signal_handlers() {
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART; // Automatically restart interrupted system calls like fgets

    if (sigaction(SIGINT, &sa, NULL) < 0) {
        perror("sigaction registration failed");
        exit(EXIT_FAILURE);
    }
}

void execute_command(char *line) {
    line = trim_space(line);
    if (strlen(line) == 0) return;

    char *args[MAX_ARGS];
    int arg_count = 0;
    char *token = strtok(line, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
    args[arg_count] = NULL;

    if (arg_count == 0) return;

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return;
    } else if (pid == 0) {
        // Child process resets SIGINT handler to default action (terminate)

```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Tex

UW PICO 5.09

File: signal_handler.c

```

    }
}

void execute_command(char *line) {
    line = trim_space(line);
    if (strlen(line) == 0) return;

    char *args[MAX_ARGS];
    int arg_count = 0;
    char *token = strtok(line, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
    args[arg_count] = NULL;

    if (arg_count == 0) return;

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return;
    } else if (pid == 0) {
        // Child process resets SIGINT handler to default action (terminate)
        signal(SIGINT, SIG_DFL);
        execvp(args[0], args);
        perror("Command execution failed");
        exit(EXIT_FAILURE);
    } else {
        // Track active foreground child PID
        foreground_pid = pid;
        int status;
        waitpid(pid, &status, 0);

        if (WIFSIGNALED(status) && WTERMSIG(status) == SIGINT) {
            printf("\nForeground job (PID: %d) terminated by SIGINT.\n", pid);
        }

        // Reset foreground PID once job terminates
        foreground_pid = 0;
    }
}

int main() {
    char input_buf[BUF_SIZE];
    setup_signal_handlers();

    while (1) {

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

UW PICO 5.09

File: signal_handler.c

```

        signal(SIGINT, SIG_DFL);
        execvp(args[0], args);
        perror("Command execution failed");
        exit(EXIT_FAILURE);
    } else {
        // Track active foreground child PID
        foreground_pid = pid;
        int status;
        waitpid(pid, &status, 0);

        if (WIFSIGNALED(status) && WTERMSIG(status) == SIGINT) {
            printf("\nForeground job (PID: %d) terminated by SIGINT.\n", pid);
        }

        // Reset foreground PID once job terminates
        foreground_pid = 0;
    }
}

int main() {
    char input_buf[BUF_SIZE];
    setup_signal_handlers();

    while (1) {
        printf("SignalShell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_command(input_buf);
    }

    return 0;
}

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

```
[medikondarakesh@Medikondas-MacBook-Air ~ % nano signal_handler.c
medikondarakesh@Medikondas-MacBook-Air ~ % nano signal_handler.c
gcc -g -Wall signal_handler.c -o signal_handler
./signal_handler
# Example interactive triggers:
# Press Ctrl+C at empty prompt
# SignalShell > sleep 30 (then press Ctrl+C)
# SignalShell > exit

# Valgrind check:
[valgrind --leak-check=full ./signal_handler
```

